# D4RT: Reconstructing Dynamic Scenes by Querying Points in Space and Time

Anonymous CVPR submission

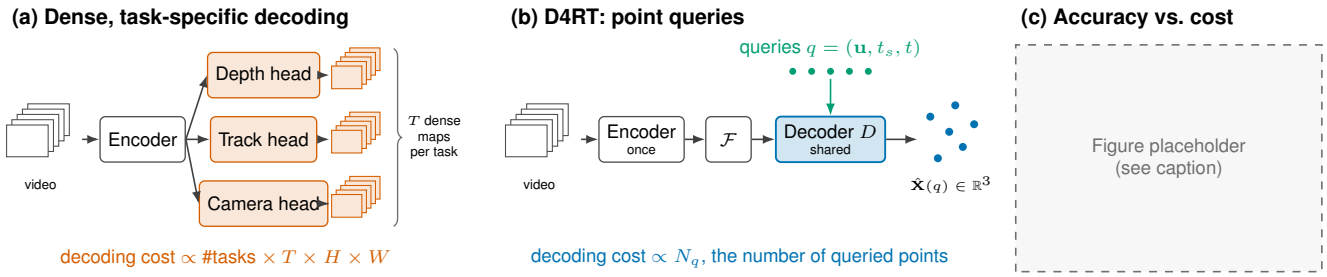



Figure 1. **Decoding on demand.** (a) Existing feedforward 4D models decode a dense map for every frame with a separate head (orange) for every task. (b) D4RT encodes the video once and answers independent point queries (green) with one shared decoder (blue), so depth, correspondence and cameras all come from the 3D positions of queried points. (c) **[TODO: Figure plan: scatter plot, $x$ = inference time per video (s, log scale), $y$ = *e.g.* video depth AbsRel or a 4D reconstruction score, one marker per method, marker size = peak memory; D4RT in blue, baselines in their fixed colours (`plot_style.py`). Same benchmark, video length and resolution as Tab. 4. Takeaway for the caption: "D4RT is more accurate than … while running $N\times$ faster".]**

## Abstract

*Recovering the geometry and motion of a dynamic scene from a single video requires depth, spatio-temporal correspondence and camera parameters at the same time. Feedforward models now predict these quantities in one pass, but they decode a dense output for every frame and attach a separate decoder to every task, so their cost grows with the length of the video and their design grows with the number of tasks. We present D4RT, a feedforward transformer that infers all three quantities from one video through a single decoding interface. D4RT encodes the video once and then answers independent queries, each asking for the 3D position of one point at one moment in time. Because depth, correspondence and camera parameters all follow from such queries, the model decodes only the points a task needs and shares one lightweight decoder across tasks. This design makes D4RT efficient to train and* ***[TODO: inference speed or memory,* e.g. *"$N\times$ faster than the strongest feedforward baseline"]****. D4RT outperforms prior methods across a wide spectrum of 4D reconstruction tasks, including* ***[TODO: one headline result with benchmark, metric and baseline]****.*

## 1. Introduction

Consider a handheld video of a person walking a dog through a park. Reconstructing this scene in 4D means recovering, for every pixel, where its surface point lies in 3D, where that point moves as the person and the dog walk, and where the camera stood when it observed the point. Classical pipelines obtain these quantities by optimizing each video separately **[TODO: cite per-video optimization methods]**. Feedforward networks have since learned to predict depth, point maps, camera poses and tracks directly from images in a single forward pass **[TODO: cite static feedforward reconstruction]**, and recent models extend this approach from static scenes to dynamic videos **[TODO: cite dynamic feedforward 4D methods]**.

The remaining cost of these models lies in how they decode. Existing feedforward 4D models decode a dense map for every frame, such as a depth map, a point map or a tracking field, and they produce each output with its own task-specific decoder **[TODO: cite]**. Dense decoding makes the cost grow with the number of frames times the number of pixels, even when a task needs only a sparse set of points, for example a few tracks followed through a long video. Task-specific decoders add a second cost, because every new quantity brings a new head, a new loss and a new balance between them. The main reason is that these mod-

els tie the format of the output to the task. A model that decodes every 4D quantity densely pays for answers that no task asked for.

Depth, correspondence and camera parameters are different readings of one quantity: the 3D position of a point at a moment in time. Depth is the position of a pixel's surface point at the time the pixel was observed, and correspondence is the position of the same point at another time, while the camera parameters relate these positions across frames **[TODO: confirm how camera intrinsics and extrinsics are obtained from point queries]**. We therefore present D4RT, a feedforward transformer built around this single question, which encodes a video once into a **[TODO: name of the scene representation]** and decodes the 3D position of any queried point at any queried time through one lightweight decoder (Fig. 1). Queries are independent of one another, so D4RT decodes only the points a task requires, and one decoder serves depth, correspondence and camera estimation alike.

This decoding interface is the main difference from prior feedforward 4D reconstruction. In contrast to **[TODO: cite closest dense feedforward 4D model]**, which decodes a full map per frame for each task, D4RT has neither a dense per-frame decoder nor a task-specific head, and its decoding cost grows with the number of queries rather than with the number of frames and pixels. **[TODO: one sentence on a second closest work and its checkable difference, *e.g.* input, assumption or output.]**

Decoding on demand makes D4RT efficient to train and to run: **[TODO: training cost compared with the strongest baseline, *e.g.* GPU hours or memory]** and inference is **[TODO: speed number]** faster than **[TODO: baseline]** on **[TODO: video length or resolution]**. D4RT also outperforms prior methods across a wide spectrum of 4D reconstruction tasks, reducing **[TODO: metric]** on **[TODO: benchmark]** from **[TODO: ]** to **[TODO: ]** and improving **[TODO: second headline result]**. These results show that a single point query, asked where and when it is needed, recovers depth, motion and cameras from one video.

## 2. D4RT: Reconstruction by Point Queries

Given a video of $T$ frames $\mathcal{V} = \{I_t\}_{t=1}^{T}$, D4RT recovers per-pixel depth, spatio-temporal correspondence and the full camera parameters of every frame. The model has two parts (Fig. 2): a transformer encoder that turns the whole video into a scene representation once, and a single lightweight decoder that answers queries about the 3D position of individual points at individual times. All three outputs are read from answers to these queries (Sec. 2.4), so the model contains no dense per-frame decoder and no task-specific head.

### 2.1. Problem formulation and notation

For a pixel $\mathbf{u} \in \mathbb{R}^2$ observed in frame $t_s$, let $\mathbf{X}(\mathbf{u}, t_s \mid t) \in \mathbb{R}^3$ denote the 3D position of its surface point at time $t$. This single quantity contains the three tasks. Depth is $\mathbf{X}(\mathbf{u}, t_s \mid t_s)$ expressed in the camera of frame $t_s$; correspondence is the trajectory $\{\mathbf{X}(\mathbf{u}, t_s \mid t)\}_t$ of the same point over time; the camera parameters of frame $t$ relate these positions to the image **[TODO: state exactly how intrinsics $K_t$ and extrinsics $(R_t, \mathbf{t}_t)$ are obtained]**. We write a query as

$$q = \big(\mathbf{u},\, t_s,\, t,\, \cdot\big), \tag{1}$$

where $\cdot$ stands for **[TODO: other query fields, *e.g.* the reference coordinate frame]**, and D4RT learns the map $q \mapsto \hat{\mathbf{X}}(q)$. **[TODO: the coordinate frame in which X is expressed: world, first camera, or a queried camera.]**

### 2.2. Encoding the video once

Dense decoders repeat their work for every frame and every task after the encoder. D4RT instead concentrates the cost in a single encoding pass that is shared by every subsequent query. A transformer encoder $E$ maps the video to a scene representation

$$\mathcal{F} = E(\mathcal{V}), \tag{2}$$

which holds what the decoder needs to locate any point at any time. **[TODO: tokenization of the video (patch size, spatio-temporal tokens), the attention pattern (*e.g.* frame-wise and global), and the size and form of $\mathcal{F}$.]** Because $\mathcal{F}$ is computed once per video, its cost is paid once no matter how many points are later queried.

### 2.3. Unified decoding by point queries

The core of D4RT is a decoder $D$ that answers one query at a time from the shared representation:

$$\hat{\mathbf{X}}(q) = D\big(\gamma(q), \mathcal{F}\big), \tag{3}$$

where $\gamma(q)$ embeds the query **[TODO: query embedding, *e.g.* positional encoding of $\mathbf{u}$ and $t$, plus local image features]**. **[TODO: decoder architecture, *e.g.* cross-attention from the query token to $\mathcal{F}$, number of layers.]** Queries do not attend to one another, so any subset of points can be decoded in parallel and the decoding cost is linear in the number of queries $N_q$ rather than in $T \times H \times W$. A tracking task that follows a few hundred points through a long video therefore decodes a few hundred queries, while dense depth for one frame decodes $H \times W$ queries **[TODO: confirm, or state the resolution at which dense outputs are queried]**.

The same decoder serves all tasks. A dense model needs one head per output format; D4RT needs only one format, a 3D position, and varies the query to obtain each task. Adding a new quantity derived from point positions therefore adds no parameters.

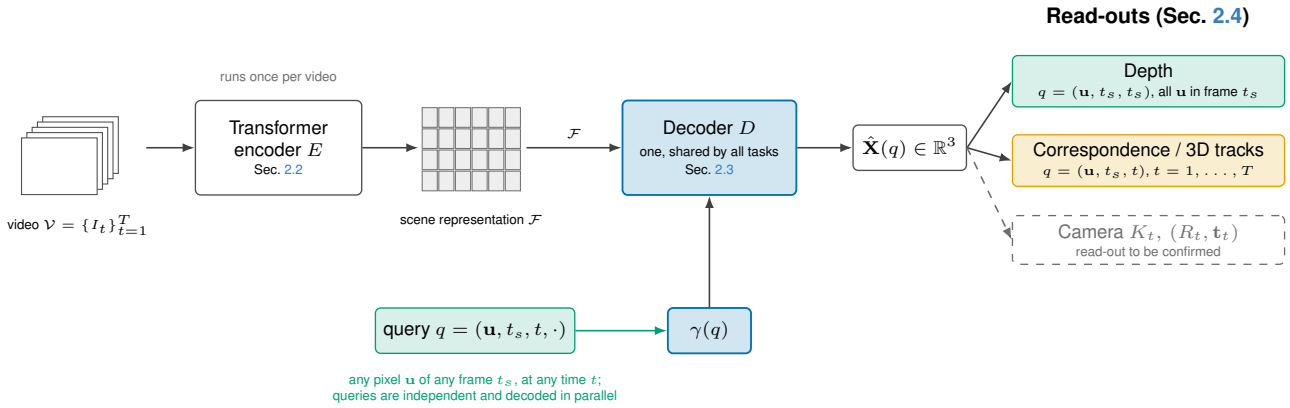



Figure 2. **Overview of D4RT.** A transformer encoder turns the video into a scene representation $\mathcal{F}$ once. Each query (green) names a pixel $\mathbf{u}$ of a source frame $t_s$ and a target time $t$, and the shared decoder (blue) returns the 3D position of that surface point at time $t$. Depth (green) and 3D tracks (yellow) are different sets of queries, so no task needs its own head; the camera read-out (dashed) is drawn once its mechanism is confirmed. **[TODO: Figure plan: replace the encoder and decoder boxes with the real architecture (tokenization, attention pattern, how $\gamma(q)$ attends to $\mathcal{F}$), add the extra query fields, and replace the blank input frames with frames from Fig. 4.]**

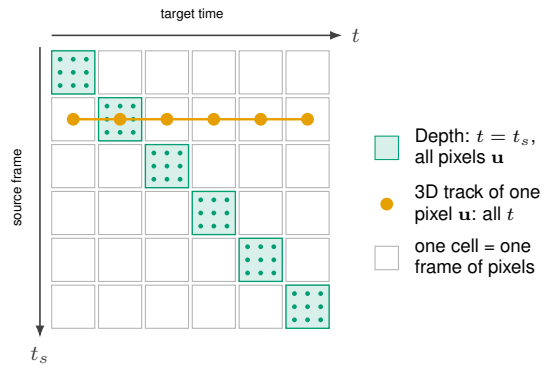



Figure 3. **Every task is a set of queries.** Each cell holds the queries $(\mathbf{u}, t_s, t)$ for all pixels of source frame $t_s$ at target time $t$. Depth uses the diagonal (green), and a 3D track uses one pixel across a row (yellow); D4RT decodes only the queries a task asks for, instead of all $T \times T$ cells. **[TODO: Figure plan: add the camera read-out once confirmed; if the query also names a reference camera, add it as a third axis or a note.]**

## 2.4. Reading out depth, correspondence and cameras

Each task is a particular set of queries (Fig. 3).

**Depth.** For frame $t_s$ we query every pixel at its own time, $q = (\mathbf{u}, t_s, t_s, \cdot)$, and take the $z$ coordinate of $\hat{\mathbf{X}}$ in the camera of frame $t_s$.

**Correspondence and 3D tracking.** For a point $\mathbf{u}$ in frame $t_s$ we query all target times $t \in \{1, \ldots, T\}$, which yields its 3D trajectory; projecting the trajectory with the camera of each frame gives 2D correspondence. **[TODO: how occlusion or visibility is predicted, if at all.]**

**Camera parameters.** **[TODO: mechanism: whether intrinsics and extrinsics are estimated in closed form from queried point sets (*e.g.* a least-squares fit between points expressed in camera and world coordinates), or predicted by another query type.]**

## 2.5. Training and inference

**Objective.** **[TODO: loss on predicted 3D positions (*e.g.* L1 or confidence-weighted regression), any normalisation of scene scale, and any auxiliary losses.]**

**Query sampling.** Because queries are independent, training supervises only a sampled subset of points per video rather than full dense maps, which keeps the decoding cost of each training step bounded. **[TODO: number of queries per video and the sampling strategy, *e.g.* uniform over pixels, biased to moving regions, or across time gaps.]**

**Training data.** **[TODO: datasets and the supervision each provides: depth, tracks, cameras.]**

**Inference.** At test time the encoder runs once per video, and each task issues its own queries to the decoder (Sec. 2.4). **[TODO: maximum video length and resolution; how longer videos are handled.]**

## 2.6. Discussion

Dense feedforward 4D models **[TODO: cite]** couple the output format to the task: each head decodes one map per

frame. D4RT decouples the two. The encoder holds the scene, and the decoder is a function that can be evaluated at any point and time, in the spirit of coordinate-based neural fields **[TODO: cite]** but predicted feedforward from the video instead of optimized per scene. What stays the same as in dense models is the supervision: D4RT is trained on the same kinds of depth, track and camera labels **[TODO: confirm]**, applied to sampled queries.

## 3. Experiments

We evaluate D4RT on the three outputs it reconstructs from one video: depth (Sec. 3.2), 3D correspondence and tracking (Sec. 3.3) and camera parameters (Sec. 3.4). We then measure the training and inference cost that the query-based decoder removes (Sec. 3.5) and ablate the design choices behind it (Sec. 3.6).

### 3.1. Setup

**Datasets.** **[TODO: evaluation benchmarks per task, with split and number of sequences. Video depth: . . . ; 3D tracking: . . . ; camera pose: . . . ]**

**Metrics.** Video depth: **[TODO: *e.g.* AbsRel ↓, $\delta_{1.25}$ ↑, with the alignment used (scale, or scale and shift, per sequence)]**. Tracking: **[TODO: *e.g.* APD ↑, EPE ↓]**. Camera: **[TODO: *e.g.* ATE ↓, RPE translation ↓, RPE rotation ↓, intrinsics error ↓]**.

**Baselines.** **[TODO: feedforward and optimization-based methods per task, and how they were run (official code and checkpoints, same input resolution and video length).]**

**Implementation.** **[TODO: encoder size, number of queries per training video, training data mixture, optimizer, iterations, hardware.]**

### 3.2. Video depth estimation

D4RT **[TODO: claim first, *e.g.* "achieves the lowest AbsRel on all $N$ benchmarks"]** (Tab. 1), with the largest margin on **[TODO: benchmark with the most motion]**. **[TODO: one sentence on why, tied to the method, *e.g.* that depth is decoded from a representation shared with motion.]** Fig. 4 shows that **[TODO: qualitative takeaway]**.

### 3.3. 3D correspondence and tracking

D4RT **[TODO: claim first]** (Tab. 2). Because each track is a set of independent queries over time (Sec. 2.4), D4RT tracks points without decoding the frames in which they are not needed; **[TODO: reason for the accuracy gain, if any]**. Fig. 5 shows tracks of points on moving objects over full videos.

Table 1. **[TODO: takeaway first]**. Video depth on **[TODO: benchmarks]**. Best in **bold**; red entries are to be filled.

| | | Bench. A | | Bench. B | |
|---|---|---|---|---|---|
| Method | FF | AbsRel ↓ | $\delta_{1.25}$ ↑ | AbsRel ↓ | $\delta_{1.25}$ ↑ |
| Baseline 1 | | ? | ? | ? | ? |
| Baseline 2 | | ? | ? | ? | ? |
| D4RT (ours) | ✓ | ? | ? | ? | ? |

Table 2. **[TODO: takeaway first]**. 3D tracking on **[TODO: benchmarks]**. Best in **bold**; red entries are to be filled.

| | Bench. A | | Bench. B | |
|---|---|---|---|---|
| Method | APD ↑ | EPE ↓ | APD ↑ | EPE ↓ |
| Baseline 1 | ? | ? | ? | ? |
| D4RT (ours) | ? | ? | ? | ? |

Table 3. **[TODO: takeaway first]**. Camera estimation on **[TODO: benchmark]**. Best in **bold**; red entries are to be filled.

| Method | ATE ↓ | RPE-t ↓ | RPE-r ↓ | Intr. ↓ |
|---|---|---|---|---|
| Baseline 1 | ? | ? | ? | ? |
| D4RT (ours) | ? | ? | ? | ? |

Table 4. **[TODO: takeaway first]**. Measured on **[TODO: hardware, video length, resolution]**.

| Method | Params | Train ↓ | Time (s) ↓ | Mem. (GB) ↓ |
|---|---|---|---|---|
| Baseline 1 | ? | ? | ? | ? |
| D4RT (ours) | ? | ? | ? | ? |

### 3.4. Camera parameter estimation

D4RT **[TODO: claim first, *e.g.* "recovers camera poses and intrinsics more accurately than methods that . . . "]** (Tab. 3). **[TODO: one sentence on why.]**

### 3.5. Efficiency and scalability

The query-based decoder makes D4RT **[TODO: $N\times$]** faster at inference than **[TODO: baseline]** on **[TODO: video length and resolution]** (Tab. 4), because the decoder evaluates only the queried points instead of a full map per frame and task. Training is **[TODO: claim with number, *e.g.* GPU hours or memory per step]**. Fig. 6 plots runtime and memory against video length and against the number of queries; **[TODO: takeaway, *e.g.* decoding cost grows linearly with the number of queries]**.

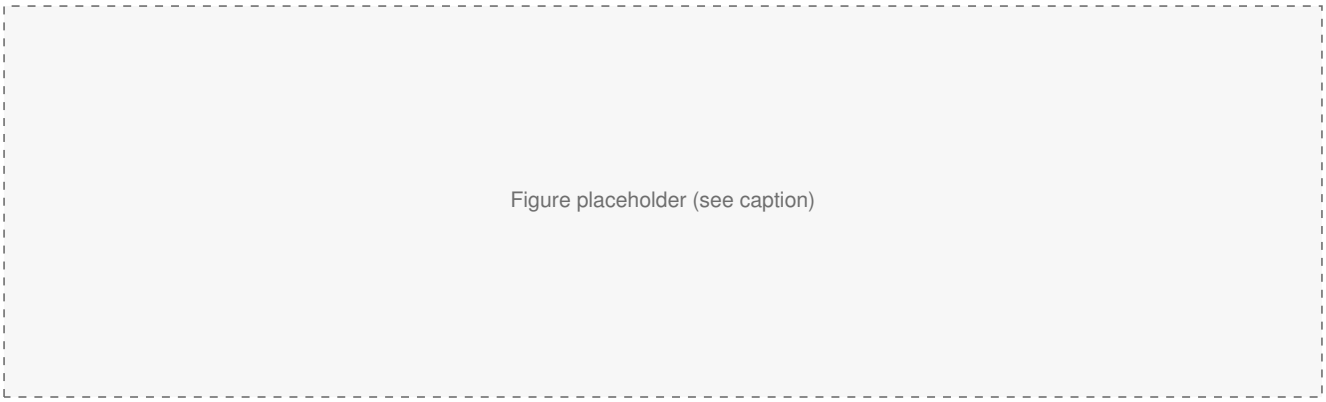



Figure 4. **Qualitative 4D reconstruction on dynamic videos. [TODO: Figure plan: 3 rows (videos) × 4 columns. Columns: input frames (3 frames, stacked); fused 4D point cloud from [TODO: strongest dense baseline]; fused 4D point cloud from D4RT; ground truth or a novel viewpoint of D4RT. Pick one video with fast object motion, one with large camera motion, one long video. Draw recovered camera frusta along the trajectory, and color the points by time. Use the same viewpoint and point size for all methods; highlight with a box where the baseline smears moving parts or duplicates geometry. Takeaway sentence for the caption: "D4RT keeps moving objects sharp and cameras consistent, where . . . ".]**

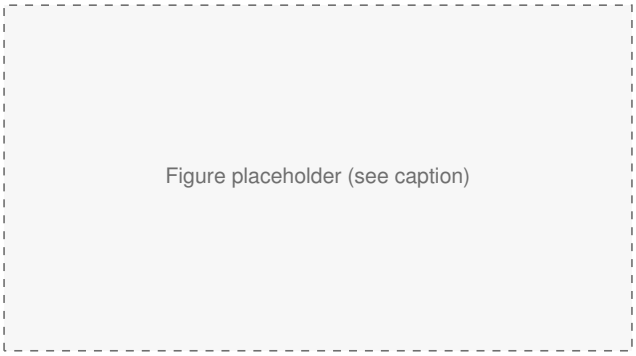



Figure 5. **3D tracks from point queries. [TODO: Figure plan: 2 videos × 2 columns ([TODO: baseline], D4RT). Show the first frame with about 30 query pixels on moving objects, and their 3D trajectories over the whole video drawn in the reconstructed scene, colored by time. Add one inset where a point is occluded for part of the video. Takeaway sentence: tracks stay on the object over the full clip, without decoding dense maps for the other frames.]**

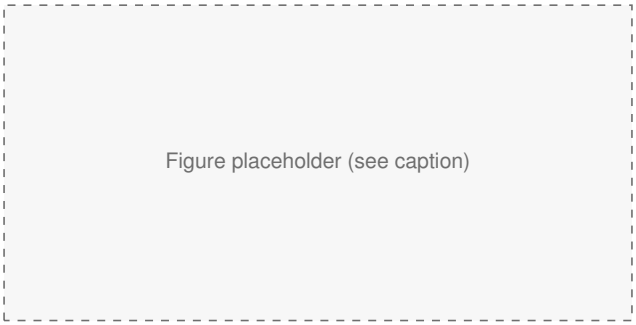



Figure 6. **Cost scales with the number of queries. [TODO: Figure plan: two line plots side by side, from the measurements behind Tab. 4. Left: inference time (s, log scale) against video length $T$ for D4RT and the dense baselines, at fixed resolution. Right: D4RT decoding time and memory against the number of queries $N_q$, with a dashed horizontal line for the cost of decoding all $T \times H \times W$ points densely. Same hardware for all curves. Takeaway sentence: D4RT decoding grows linearly in $N_q$, so sparse tasks cost a fraction of dense decoding.]**

### 3.6. Ablation study

Each ablation answers one question about the design (Tab. 5).

**Does one shared decoder match task-specific decoders?** *Task-specific heads* replaces the unified decoder with one decoder per output. **[TODO: finding.]**

**Does query-based decoding lose accuracy compared with dense decoding?** *Dense decoder* replaces point queries with a dense per-frame decoder of similar size. **[TODO: finding on accuracy and cost.]**

**Does joint training across tasks help each task?** *Single-task* trains D4RT on one output at a time. **[TODO: finding.]**

**How many queries does training need?** We vary the number of queries per training video. **[TODO: finding.]**

### 3.7. Limitations

**[TODO: concrete failure modes with their cause, for example very long videos beyond the encoder's context,**

Table 5. **[TODO: takeaway first]**. Ablations on **[TODO: benchmark]**.

| Variant | Depth | Track | Camera | Time ↓ |
|---|---|---|---|---|
| Task-specific heads | ? | ? | ? | ? |
| Dense decoder | ? | ? | ? | ? |
| Single-task | ? | ? | ? | ? |
| D4RT (full) | ? | ? | ? | ? |




Figure 7. **Failure cases.** **[TODO: Figure plan: 2 or 3 examples, each with input frame and D4RT output, with the error region circled. Candidates (to be confirmed from your results): very long videos beyond the encoder context, thin or fast moving structures, textureless regions. One sentence per example states the cause.]**

**thin or fast moving structures, or scenes with little texture, with a figure.]** Examples are shown in Fig. 7.